

Using VS Code in This Course

VS Code is the standard editor for this course. You will usually use it to open a course repository, run code, work with notebooks, use the terminal, and manage Git changes in one place.

What VS Code is

Visual Studio Code is a free code editor with built-in support for editing, terminals, source control, debugging, and extensions.

In practice, that means you can use one tool to:

- open your course folder
- edit Python files and notebooks
- run terminal commands
- use Git and GitHub
- install extensions for class workflows

Why use VS Code

Students usually care about a few practical benefits:

- It keeps editing, notebooks, terminal work, and Git in one place.
- It works well with Python and Jupyter.
- It helps you pick the correct Python interpreter and notebook kernel.
- It makes it easier to stay inside the correct course folder while you work.
- It can also connect to a remote machine if you need to work on a server or lab system.

How VS Code fits into this course

In this course, the standard workflow is:

1. Open the course repository folder in VS Code.
2. Open the integrated terminal.
3. Run `uv sync` in that folder.
4. Select the Python interpreter for that repository.
5. Select the notebook kernel for that repository.
6. Run scripts, tests, and notebooks.
7. Use built-in Git tools or terminal Git commands as needed.

The most important habit is this: open the repository folder, not just a single file.

If you open only a file instead of the full project folder:

- the terminal may start in the wrong place
- uv commands may run in the wrong directory
- Git may not recognize the repository
- notebooks may use the wrong environment

For package management and Git to work smoothly, your working directory in VS Code should be the root folder of the course repository.

Installing VS Code

Because students use different operating systems, the easiest approach is to use the official VS Code download page and choose the installer or package that matches your system.

General process:

1. Go to the official VS Code download page.
2. Download the installer or package for your operating system and hardware.
3. Run the installer and follow the prompts.
4. Open VS Code after installation.
5. If your platform supports it, enable the code command in your terminal PATH so you can open a folder with `code ..`

After installation, it is helpful to confirm that VS Code opens normally and can open a folder on your computer.

Installing extensions

Extensions add support for the tools and workflows used in the course.

To install extensions:

1. Open VS Code.
2. Open the Extensions view.
3. Search for the extension by name.
4. Select the extension.
5. Choose Install.

Standard extensions for most students:

- Python
- Jupyter

Recommended if you work on a remote machine, server, or lab system:

- Remote - SSH

Additional suggested extensions:

- Ruff
- YAML
- Markdown All in One
- Rainbow CSV
- GitHub Pull Requests and Issues
- GitHub Copilot, if approved or used in the course
- GitHub Copilot Chat, if approved or used in the course

Getting started

Open the correct folder

This is the most important setup step in VS Code.

Open the repository folder itself, not just an individual file inside it.

Good ways to do that:

- Use **File > Open Folder** and choose the course repository folder.
- If your terminal is already inside the repository, run `..`

Your working directory should be the folder that contains files such as:

- `.git/`
- `pyproject.toml`
- `uv.lock`
- `notebooks/`
- `src/`

When you open the correct folder:

- the terminal usually starts in the right place
- Git recognizes the repository
- `uv sync` applies to the correct project
- VS Code is more likely to detect the correct environment

Open the integrated terminal

After opening the repository folder:

1. Open the integrated terminal in VS Code.
2. Confirm you are in the repository root.
3. Run course commands from there.

Typical commands:

```
uv sync
uv run python src/setup_check.py
git status
```

If a command fails unexpectedly, first check whether the terminal is in the correct working directory.

Open the Extensions view

Use the Extensions view to install or manage course extensions.

You will usually install the Python and Jupyter extensions first, then add Remote - SSH only if you need remote development.

Select a Python interpreter

Once the repository is open:

1. Open the Command Palette.
2. Run Python: Select Interpreter.
3. Choose the interpreter that belongs to the course repository environment.

If the course uses uv, make sure you select the interpreter created for that project rather than a global Python installation.

Select a notebook kernel

For notebooks:

1. Open the notebook file.
2. Use the kernel picker in the notebook editor.
3. Choose the kernel that matches the course repository environment.

The interpreter and notebook kernel should point to the same project environment whenever possible.

Common tasks in VS Code

Running a Python file

1. Open the repository folder.
2. Open the integrated terminal.
3. Make sure the terminal is in the repository root.
4. Run the script from there.

Example:

```
uv run python src/setup_check.py
```

Running notebook cells

1. Open the notebook.
2. Confirm the correct notebook kernel is selected.
3. Run cells one at a time or use the notebook controls to run all cells.

If notebook imports fail, double-check the kernel before changing anything else.

Using the terminal

The integrated terminal is where you will usually:

- run `uv sync`
- run Python scripts
- run tests
- run Git commands

Try to keep the terminal rooted in the course repository folder. That keeps package management and Git behavior much more predictable.

Viewing source control changes

VS Code has built-in source control support.

You can use it to:

- see changed files
- stage files
- write commit messages
- commit changes
- review which files are untracked

This is helpful, but terminal Git commands are also fine if you prefer them.

Staging and committing

Typical workflow:

1. Open the repository folder.
2. Check the Source Control view or run `git status` in the terminal.
3. Stage the files you want to include.
4. Write a commit message.
5. Commit the changes.
6. Push them when appropriate.

If Git behavior seems odd, verify again that VS Code is opened at the repository root.

Connecting to a remote host with Remote - SSH

Use Remote - SSH only if you need to work on a remote machine.

Basic first-use workflow:

1. Make sure you can SSH into the remote machine from a terminal.
2. Install the Remote - SSH extension.
3. Open the Command Palette.
4. Run Remote-SSH: Connect to Host....
5. Choose or enter the host you want to use.
6. Open the remote project folder after the connection succeeds.

This requires:

- a working SSH client on your local machine
- access to a reachable SSH server
- correct login credentials or SSH keys

Suggested extensions

Standard for most students

- Python Adds Python language support, interpreter selection, running, and debugging support.
- Jupyter Adds notebook support inside VS Code.

Recommended for some workflows

- Remote - SSH Helpful if you work on a server, lab machine, or another remote system.
- Ruff Helpful for formatting and linting if the course uses it.
- YAML Helpful for configuration files.
- Markdown All in One Helpful for editing Markdown files.
- Rainbow CSV Helpful when looking at CSV files directly.

If approved or used in the course

- GitHub Pull Requests and Issues
- GitHub Copilot
- GitHub Copilot Chat

Common problems and fixes

VS Code opened a file instead of the project folder

Symptoms:

- the Explorer shows only one file
- Git features seem missing
- uv or Git commands run in the wrong place

What to try:

- Close the file-only view.
- Reopen VS Code with **File > Open Folder**.
- Choose the repository root folder.
- Recheck the terminal working directory.

Python interpreter mismatch

Symptoms:

- imports fail even though packages should be installed
- code works in one place but not another

What to try:

- **Run Python: Select Interpreter**.
- Choose the interpreter for the course repository environment.
- Re-run the script after selecting it.

Notebook kernel mismatch

Symptoms:

- notebook cells fail on imports
- notebook output does not match terminal behavior

What to try:

- Open the notebook kernel picker.
- Select the kernel for the course repository environment.
- Restart the kernel and run the notebook again.

Extension is installed but not active

Symptoms:

- expected commands do not appear
- notebooks or Python features are missing

What to try:

- Make sure the extension is installed.
- Reload or restart VS Code.
- Check whether the extension is disabled for the workspace.

code command not found

Symptoms:

- `code` . fails in the terminal

What to try:

- Open VS Code normally first.
- Use the platform-specific option to add the `code` command to your PATH if needed.
- Restart the terminal and try again.

Remote - SSH problems

Symptoms:

- host connection fails
- VS Code cannot connect or set up the remote session

What to try:

- Test the SSH connection in a normal terminal first.
- Confirm the hostname, username, and authentication method are correct.
- Make sure the remote machine is reachable.
- If the problem continues, ask for help with the full error text.

When to ask for help

Ask for help early if VS Code is getting in the way of your work. When you ask, include:

- operating system
- VS Code version
- extension names involved
- exact steps you tried
- full error message as text
- whether the issue is about local setup, notebooks, Git, or Remote - SSH

That information makes it much easier for someone to help you quickly.